

Seminar Track - Stage 01 - Tool Survey Proposal

Topic: T05 - Performance Testing

Group: Team 09

Course: Software Testing (CS423/CSC13003)

Lecturers: Dr. Lâm Quang Vũ

Instructors / TAs: MSc. Trương Phước Lộc, MSc. Hồ Tuấn Thanh

System Under Test (SUT): EShop (Node.js + SQLite)

Team Members:

Student ID	Full Name
23127006	Trần Nguyễn Khải Luân
23127128	Nguyễn Thành Tiến
23127179	Nguyễn Bảo Duy
23127184	Lê Phạm Kiều Duyên

June, 2026

1. Introduction

This proposal is prepared for **Stage 01 (S1) - Topic claim & tool survey** under the Software Testing Seminar Track. The team focuses on **T05 - Performance Testing** to simulate load, stress, and spike workloads on the local EShop backend. The primary objective is to evaluate and pair a reliable traditional testing tool with an AI-augmented solution to optimize script generation from realistic log data.

2. Theoretical Foundations of Web Performance Testing

2.1. Core Profiles and Web-Specific Characteristics

Performance testing evaluates the responsiveness, stability, and scalability of a system under a given workload rather than verifying functional correctness. For the EShop Node.js + SQLite web backend, testing focuses on three load profiles and web dynamics:

- **Load Testing:** Evaluates system behavior under expected normal and peak conditions to verify baseline targets.
- **Stress Testing:** Pushes the application beyond normal operational capacity to identify structural breaking points and recovery behaviors.

- **Spike Testing:** Measures stability during sudden, dramatic traffic surges (e.g., ramping from 50 to 500 Virtual Users (VUs) within a 30-second window).
- **HTTP & Session Dynamics:** Web performance tools must handle stateless HTTP/HTTPS requests, persist session cookie states, and maintain isolated user contexts across dynamic transactions.
- **Backend Concurrency Locks:** While the Node.js layer handles high-concurrency asynchronous execution efficiently, the embedded SQLite database operates on a strict single-connection writing paradigm, making it highly vulnerable to thread contention and table locking under sudden write surges.

2.2. Core Performance Engineering Principles

To ensure automated execution generates valid data, three strict engineering constraints are enforced:

- **Workload Modeling:** Discards standalone endpoint repetition in favor of a chained mixed-workload model reflecting true consumer distribution patterns (e.g., search-heavy and checkout-bursty flows).
- **Pacing and Think Time:** Integrates calculated delays between sequential HTTP requests to accurately replicate human reading and clicking intervals, preventing artificial request flood corruption.
- **Service Level Objectives (SLOs):** System health is measured using strict statistical metrics instead of broad averages:
 - **Percentile Latency:** Tracks maximum response time boundaries at p50, p95, and p99 markers (e.g., target p95 < 500 ms).
 - **Error Rate:** Measures the total percentage of failed, invalid, or dropped HTTP transactions (target < 1%).

2.3. Standard Performance Testing Workflow

The execution lifecycle follows a clean, 4-stage automated pipeline:

[1. Identify Environment & SLOs] → [2. Design Workload Model] → [3. Execute Automated Load] → [4. Telemetry Triage & Report]

1. **Identify Environment & SLOs:** Establish local baseline environment hardware limits and lock down target performance goals.
2. **Design Workload Model:** Map out precise percentage distributions of typical user journeys across specific functional modules.
3. **Execute Automated Load:** Initialize JMeter or k6 scripts to scale concurrent VUs along predefined ramp curves while logging transactional metrics.
4. **Telemetry Triage & Report:** Extract percentile metrics (p50/p95/p99), isolate system architectural bottlenecks, and evaluate formal SLO compliance.

3. Candidate Tools

3.1. Traditional Tools

- **k6 (Grafana Labs):** A developer-centric, lightweight command-line tool using JavaScript for scripting (Test-as-Code).
- **Apache JMeter:** A standard, robust open-source GUI/CLI tool with a massive plugin ecosystem based on Java.
- **Gatling OSS:** A high-performance load testing tool written in Java/Kotlin/Scala utilizing an asynchronous architecture to minimize memory footprint.

3.2. AI-Augmented Directions

- **ChatGPT / Claude / GitHub Copilot (for k6):** LLMs used to instantly translate EShop server HTTP/HAR logs into executable testing scripts.
- **Grafana k6 AI:** An AI assistant that analyzes test logs, detects anomalies, and auto-suggests thresholds from past executions.
- **Akamas (AI Optimization):** An ML-driven platform that automatically tunes JVM, database connection pools, and core parameters.

3.3. Backup Candidates

- **Apache JMeter:** Chosen as a fallback for the traditional side in case script compilation or environment issues arise with k6.
- **Gatling OSS:** A high-concurrency alternative backup ensuring compile-safe scripting capability if k6 encounters environment blockades.

4. Comparison Matrix

The candidate tools are rigorously evaluated across the baseline parameters utilizing a standardized mapping framework (**Low** → **Medium** → **High / Expert / Massive**) to ensure structural objectivity.

Part A: Financial Licensing, Learning Curve, Core Platform Fit and Community

Tool	Licence Cost	Learning Curve	EShop Fit	Community
k6 <i>(Grafana Labs)</i>	Free (OSS). Open-source under AGPLv3.	Low-Medium. Approachable via standard JavaScript. No heavy GUI overhead.	High. Natively supports multi-step user journeys, session cookies, and dynamic correlations. Lightweight Go-runtime avoids thread locks on SQLite.	High. Backed by Grafana Labs with fast-growing cloud-native adoption.
Apache JMeter	Free (OSS). Open-source under	Medium. Straightforward GUI layout for drag-and-drop script building;	Medium-High. Supports multi-step journeys and session correlation, but heavy	Massive. The industry standard with decades of production maturity,

Tool	Licence Cost	Learning Curve	EShop Fit	Community
	Apache License 2.0.	advanced logic controllers possess a steeper learning curve.	thread-per-user model limits scalability on local resource-constrained environments.	web tutorials, and a rich plugin ecosystem.
Gatling OSS	Free (OSS). Open-source under Apache License 2.0.	High. Requires solid programming foundations in Java, Kotlin, or Scala, alongside its reactive Domain-Specific Language (DSL).	High. Supports complex transaction chains and dynamic data injection. Asynchronous Netty architecture scales well with low resource consumption.	High. Well-established with strong documentation and broad enterprise adoption, though smaller than JMeter.
ChatGPT / Claude <i>(for k6)</i>	Paid (Free tier available).	Very Low. Natural language prompting significantly reduces coding effort and lowers barriers to creating performance test scripts.	High. Effective at transforming HTTP traces, HAR files, and browser activity logs into structured EShop user journeys.	Massive. Backed by a large global user base and rapidly growing adoption within software engineering workflows.
Grafana k6 AI	Paid (Enterprise trial available).	Low. Integrated within Grafana Cloud dashboards, reducing setup effort and simplifying performance monitoring workflows.	High. Correlates load-testing results with application and infrastructure telemetry, enabling end-to-end performance analysis.	Moderate. Strong adoption within Grafana observability users, though smaller than mainstream testing communities.
Akamas <i>(AI Optimization)</i>	Paid (Enterprise only).	High. Requires complex infrastructure setups, microservices modeling, and advanced machine learning tuning.	Low-Medium. Over-engineered for a lightweight single-connection SQLite system; designed for massive distributed targets.	Niche. Primarily adopted within enterprise performance engineering and full-stack optimization teams.

Part B: Advanced Automation, Engineering, and Intelligent Capabilities

Tool	AI Capability	Script Maintainability	CI/CD & Environment
k6 (<i>Grafana Labs</i>)	High. Clean text scripts (<code>.js</code>) are optimized for LLM generation/auditing.	Excellent. Native Test-as-Code structure. 100% readable and trackable via Git branches.	Expert. CLI-first, low-resource container execution. Seamless for GitHub Actions.
Apache JMeter	Medium-Low. JMX XML files are verbose and difficult to review manually. Complex multi-step workflows are harder for LLMs to synthesize.	Low. XML-based test plans are difficult to review and diff. Teams frequently encounter merge conflicts in Git workflows.	Medium. Supports headless execution (Non-GUI mode), but container deployment is clunky due to heavy Java dependencies.
Gatling OSS	Medium-High. Code-based scripts are generally LLM-friendly, though complex DSL constructs can still lead to generation errors.	Excellent. Native Test-as-Code structure. Clean programmatic scripts are highly readable, easy to refactor, and fully optimized for Git tracking.	High. Lightweight CLI execution makes containerization simple, though it still requires a JRE setup within automated pipelines.
ChatGPT / Claude (<i>for k6</i>)	Very High. Efficient at multi-step script generation, workload modeling, code refactoring, and test data parameterization.	Excellent. Assists with script modularization, parameterization, code cleanup, and rapid identification of syntax or logic issues.	N/A. Functions as a development assistant rather than a load-testing engine; does not execute tests natively.
Grafana k6 AI	High. Provides intelligent anomaly detection, trend analysis, and alerting capabilities to help identify potential performance regressions.	Medium. Focuses primarily on metrics analysis and visualization rather than direct test-script maintenance.	High. Integrates smoothly with cloud-native CI/CD pipelines for continuous performance monitoring and trend analysis.
Akamas (<i>AI Optimization</i>)	Very High. Uses autonomous optimization and reinforcement-learning techniques to explore and tune infrastructure performance parameters.	Medium. Focuses on optimization policies and environment configuration rather than application load-test scripts.	High. Deeply integrates with enterprise DevOps pipelines to automate continuous performance tuning (CO) across cloud environments.

5. Recommended Pick & Rationale

- **Selected Core Pair: k6 (Traditional)** integrated with **ChatGPT / Claude (AI-Augmented Direction)**.
- **Selected Backup Tool: Apache JMeter.**

Rationale for Selection

- **Tech-Stack Alignment (JavaScript):** The EShop backend is built with Node.js, and k6 uses standard JavaScript for its test scripts. Since everyone on the team already knows JavaScript, we do not need to waste time learning a new programming language. This allows the whole team to write, review, and understand the test scripts immediately.
- **Best Fit for AI Assistance:** Since this seminar focuses on AI-assisted testing, k6 is a strong fit. LLMs such as ChatGPT and Claude are highly effective at transforming HTTP traces and HAR files into maintainable JavaScript-based k6 scripts. While AI can also generate JMeter test plans, XML-based .jmx files are generally harder to review, modify, and troubleshoot than code-based scripts.
- **Lightweight Footprint and Database Safety:** EShop uses an embedded SQLite database, which supports limited concurrent write operations and can become a bottleneck under heavy load. k6's lightweight Go-based architecture enables efficient load generation while providing precise control over request pacing and think times, helping us evaluate realistic system limits.
- **Reliable Backup Plan:** Apache JMeter serves as a dependable fallback option. Its mature ecosystem, visual test-plan designer, and extensive documentation allow test scenarios to be created and executed quickly if unexpected integration challenges arise.

6. Proposed End-to-End Test Scenario Strategy

To strictly avoid isolated endpoint testing and satisfy the course requirements, Team 09 designs a sequential, chained User Journey that mimics realistic multi-step transactions on the EShop platform. This strategy evaluates session persistence, dynamic request dependency handling, and database write contention on the Node.js + SQLite backend.

6.1. The Chained EShop User Journey (User Flow)

Instead of executing standalone HTTP requests, each Virtual User (VU) will execute a continuous transaction pipeline structured as follows:

[Step 1: Browse Home] → [Step 2: Search Products] → [Step 3: View Product Details] → [Step 4: Add to Cart] → [Step 5: Checkout Burst]

1. **Step 1: Browse Home (GET /)** – Enter the application storefront, initialize session contexts, and download static configurations.
2. **Step 2: Search Products (GET /api/products?search=...)** – Simulate multiple product searches using randomized keywords to test read query efficiency.
3. **Step 3: View Product Details (GET /api/products/{id})** – Inspect specific item fields and dynamically pull operational attributes.
4. **Step 4: Add to Cart (POST /api/cart)** – Append items to the active user session cart, requiring cookie and token tracking.
5. **Step 5: Checkout/Purchase (POST /api/checkout)** – Execute the final transactional write order, forcing SQLite to process structured modifications.

6.2. Mixed-Workload Model Distribution

To fulfill the requirement for a "**search-heavy, checkout-bursty**" profile, VUs will not execute all steps at the same rate. The load distribution model calculates specific operational mixes per virtual user-minute:

- **Browsing & Search Actions (Steps 1, 2, 3):** Allocated **60%** of total runtime weight to simulate extensive catalog exploration.
- **Cart Interactions (Step 4):** Allocated **30%** of total runtime weight to bridge navigation with purchasing interest.
- **Checkout Volume (Step 5):** Allocated **10%** of total runtime weight, executed in concentrated bursts to deliberately stress database concurrency lockups.

6.3. Concurrency Load Profiles Execution Plan

The chosen pair (k6 + ChatGPT) will execute two distinct, time-boxed operational profiles against the local SUT infrastructure to extract comparative data:

- **Baseline Load Profile:** Scaled at a stable rate of **50 concurrent VUs** sustained over **5 minutes** to evaluate baseline p95 latency margins under normal operational bounds.
- **Spike Load Profile:** Initiated at a 50 VU baseline, then sharply ramped up from **50 to 500 concurrent VUs within a tight 30-second window** to measure structural breaking points, recovery latency, and immediate error processing behaviors.

6.4. AI-Augmented Workflow Implementation

The team will integrate AI tools systematically into this scenario via a 3-step pipeline to satisfy the mandatory AI-pairing rules:

1. **HAR Capture:** A team member manually completes the 5-step user journey on the active EShop interface, exporting the raw session into an HTTP Archive (**.har**) file logs.
2. **LLM Translation:** The raw file data is supplied directly to ChatGPT/Claude with specialized templates to automatically synthesize clean, parameterized JavaScript code matching the k6 framework schema.
3. **Human Engineering Audit:** The team audits the AI-generated script output to insert realistic pacing delays (think times), dynamic correlation variables, and ensure boundary conditions are validated cleanly before the live seminar execution.

7. AI Disclosure

- **AI Tool Utilization:** The team utilized Gemini to accelerate the initial research phase, brainstorm and define the technical characteristics of the seven candidate tools, and scaffold the structural layout of the comparison matrix.
- **Human Verification and Cross-Checking:** To maintain strict academic integrity and eliminate potential AI hallucinations, every technical claim was verified by the team. We manually cross-checked k6's specific open-source licensing model (AGPLv3) and core CLI execution commands directly on the official Grafana k6 documentation portals. Furthermore, the architectural constraints regarding the single-connection transactional writing behavior of the backend SQLite

database were cross-referenced against the T05 seminar topic brief to ensure our proposed chained workload model remains entirely realistic.